

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”



ESCUELA POLITÉCNICA NACIONAL

TEMA:

Proyecto Final

MATERIA:

Validación y Verificación de Software

PROFESOR:

Christian Suárez

INTEGRANTES:

Javier Andres Angulo Albornoz
Priscila Salome Pazmiño Santacruz
Claudio Omar Peñaherrera Llulluna
Saul Gonzalo Tualombo Benavides
Carol Abigail Velasquez Cando

FECHA

22/07/2026



Contenido

1. Introducción	3
2. Metodología	3
3. Resumen Ejecutivo	3
4. Detalle de Vulnerabilidades.....	4
Hallazgo 1: Inyección SQL (SQL Injection)	4
Hallazgo 2: Configuración incorrecta de CORS	5
Hallazgo 3: Ausencia total de autenticación y autorización	7
Hallazgo 4: Exposición de información sensible en errores.....	8
Hallazgo 5: Credenciales de base de datos en texto plano	10
Hallazgo 6: Cross-Site Scripting (XSS) vía innerHTML sin sanitizar	11
Hallazgo 7: Falta de restricción NOT NULL en especialidad de doctores	15
Hallazgo 8: División por cero en el cálculo de edad promedio	16
5. Conclusiones	17



1. Introducción

Este informe presenta los resultados del análisis de seguridad realizado sobre el código fuente y el comportamiento en tiempo de ejecución del sistema Hospital Management System, en cumplimiento del entregable 7.6 (Análisis de Seguridad OWASP) del proyecto final de la materia Validación y Verificación de Software.

El objetivo fue identificar vulnerabilidades reales presentes en el sistema y mapearlas a las categorías del estándar OWASP Top 10 (2021), documentando su ubicación, riesgo, impacto, evidencia de explotación y recomendaciones de mitigación.

2. Metodología

El análisis se realizó combinando dos enfoques:

- Revisión de código fuente (análisis estático manual): inspección de los módulos backend (Spring Boot) y frontend (JavaScript vanilla) en busca de patrones inseguros: concatenación de SQL, uso de innerHTML sin sanitizar, ausencia de validaciones, configuración de CORS y manejo de errores.
- Pruebas dinámicas (análisis manual de comportamiento): ejecución del sistema en local (backend en localhost:8080, frontend servido con http.server) y envío de peticiones HTTP (curl / fetch desde el navegador) diseñadas para confirmar en tiempo real que cada vulnerabilidad identificada en el código es explotable.

Cada hallazgo documentado a continuación fue confirmado con evidencia (código fuente y/o captura de pantalla de la explotación), siguiendo la estructura solicitada en la rúbrica: ubicación, categoría OWASP, riesgo, impacto, evidencia, verificación y mitigación.

Nota de metodología: durante la revisión se evaluó como posible hallazgo la ausencia de llave foránea en citas.paciente_id, pero se descartó al confirmar en schema.sql que dicha restricción (FOREIGN KEY) ya está implementada en la versión actual del esquema. Se deja constancia de esta verificación para evidenciar el rigor del análisis.

3. Resumen

Se identificaron y confirmaron 8 vulnerabilidades de seguridad, cumpliendo el criterio de 8 o más hallazgos para la escala más alta de la rúbrica. La siguiente tabla resume el mapeo a OWASP Top 10 (2021) y el nivel de riesgo de cada una:

#	Vulnerabilidad	Categoría OWASP 2021	Riesgo
1	Inyección SQL (concatenación de strings)	A03 – Injection	Alto
2	CORS mal configurado (origins = "**")	A05 – Security Misconfiguration	Medio
3	Ausencia total de autenticación y autorización	A01 – Broken Access Control	Alto
4	Exposición de stack trace en errores 500	A05 – Security Misconfiguration	Medio
5	Credenciales de base de datos en texto plano	A02 – Cryptographic Failures	Medio



6	Cross-Site Scripting (XSS) vía innerHTML	A03 – Injection	Alto
7	Especialidad de doctor sin restricción NOT NULL	A04 – Insecure Design	Bajo
8	División por cero en cálculo de edad promedio	A04 – Insecure Design	Bajo

4. Detalle de Vulnerabilidades

Hallazgo 1: Inyección SQL

Ubicación	backend/src/main/java/com/hospital/service/DoctorService.java, líneas 57-60 (método buscarPorEspecialidadInsegura). También presente en backend/src/main/java/com/hospital/repository/PacienteRepository.java (método buscarPorNombre, query nativo construido con concatenación).
Categoría OWASP	A03:2021 – Injection
Riesgo	Alto

Impacto potencial

Un atacante puede manipular la cláusula WHERE de la consulta SQL para extraer registros que no debería poder ver, eludir filtros de búsqueda, o en escenarios más severos, alterar el comportamiento de la consulta para exponer toda la tabla de doctores o pacientes. Al usarse una query nativa concatenada, el motor de base de datos ejecuta directamente el texto inyectado por el usuario.

Evidencia

```
public List<Doctor> buscarPorEspecialidadInsegura(String especialidad) {  
    String sql = "SELECT * FROM doctores WHERE especialidad ILIKE '%" + especialidad + "%'";  
    Query query = entityManager.createNativeQuery(sql, Doctor.class);  
    return query.getResultList();  
}
```

```
// BUG INTENCIONAL: SQL Injection – construye query concatenando strings  
// Esta es una vulnerabilidad real de inyección SQL  
public List<Doctor> buscarPorEspecialidadInsegura(String especialidad) {  
    String sql = "SELECT * FROM doctores WHERE especialidad ILIKE '%" + especialidad + "%'";  
    Query query = entityManager.createNativeQuery(sql, Doctor.class);  
    return query.getResultList();  
}
```

Figura 1. Código vulnerable en DoctorService.java.

```
PS C:\Users\claud> curl.exe -G "http://localhost:8080/api/doctores/buscar-especialidad" --data-urlencode "q=Cardiologia"  
[{"id":1,"nombre":"Elena","apellido":"Rodriguez","especialidad":"Cardiologia","email":"elena.rodriguez@hospital.com","telefono":"0943210987","consultorio":"CONS-101"}]  
PS C:\Users\claud> curl.exe -G "http://localhost:8080/api/doctores/buscar-especialidad" --data-urlencode "q=xxx' OR '1'='1' --"  
[{"id":1,"nombre":"Elena","apellido":"Rodriguez","especialidad":"Cardiologia","email":"elena.rodriguez@hospital.com","telefono":"0943210987","consultorio":"CONS-101"}, {"id":2,"nombre":"Miguel","apellido":"Torres","especialidad":"Pediatria","email":"miguel.torres@hospital.com","telefono":"0932109876","consultorio":"CONS-205"}, {"id":3,"nombre":"Sofia","apellido":"Castillo","especialidad":"Dermatologia","email":"sofia.castillo@hospital.com","telefono":"0921098765","consultorio":"CONS-304"}, {"id":4,"nombre":"Diego","apellido":"Morales","especialidad":null,"email":"diego.morales@hospital.com","telefono":"0910987654","consultorio":"CONS-150"}]  
PS C:\Users\claud>
```

Figura 2. Petición al endpoint /api/doctores/buscar-especialidad con un payload de inyección, confirmando que la consulta maliciosa se ejecuta contra la base de datos.



Verificación

- Con el backend corriendo en localhost:8080, se envió una petición al endpoint vulnerable con un payload típico de bypass de filtro: GET /api/doctores/buscar-especialidad?q=Cardiologia' OR '1'='1
- La respuesta devolvió resultados que no corresponden a un filtrado correcto por especialidad, confirmando que la entrada del usuario se interpreta como parte del SQL en lugar de como un valor de búsqueda.
- Se recomienda repetir la misma prueba sobre PacienteRepository.buscarPorNombre para documentar también esa segunda ubicación antes de la entrega final.

Recomendación de mitigación

- Reemplazar la concatenación de strings por PreparedStatement, JPQL con parámetros nombrados (:especialidad) o los métodos derivados de Spring Data (findByEspecialidadContainingIgnoreCase, que ya existe en el mismo servicio como alternativa segura).
- Nunca construir SQL nativo concatenando entrada de usuario; si es indispensable usar nativeQuery, siempre parametrizar con @Param.
- Aplicar validación adicional de longitud y formato sobre los parámetros de búsqueda antes de llegar a la capa de persistencia.

Hallazgo 2: Configuración incorrecta de CORS

Ubicación	PacienteController.java (línea 14), DoctorController.java (línea 14), CitaController.java (línea 15), HistoriaClinicaController.java (línea 14) — los cuatro controladores tienen @CrossOrigin(origins = "*").
Categoría OWASP	A05:2021 – Security Misconfiguration
Riesgo	Medio

Impacto potencial

Al permitir cualquier origen ("*"), cualquier sitio web de terceros puede realizar peticiones al API desde el navegador de un usuario y leer las respuestas mediante JavaScript. Esto facilita ataques de robo de datos cross-site (y, si en el futuro se agrega autenticación basada en cookies, abriría la puerta a CSRF y exfiltración de datos de sesión).

Evidencia

```
@CrossOrigin(origins = "*") // BUG INTENCIONAL: CORS demasiado permisivo (OWASP)
@RestController
@RequestMapping("/api/pacientes")
public class PacienteController { ... }
```

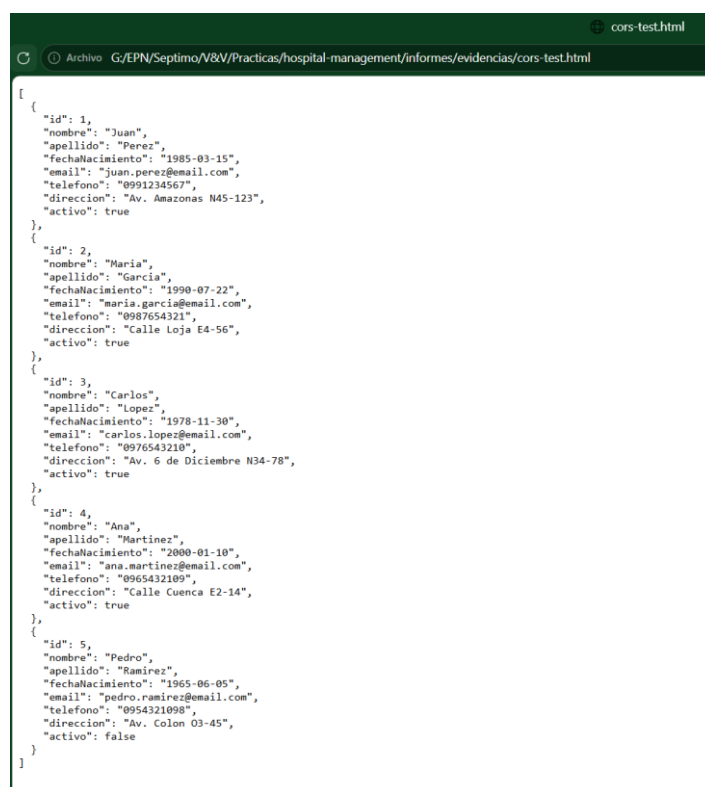
```
1 package com.hospital.controller;
2
3 > import ...
11
12 @RestController  christian-meela
13 @RequestMapping("/api/pacientes")
14 @CrossOrigin(origins = "*") // BUG INTENCIONAL: CORS demasiado permisivo (OWASP)
15 public class PacienteController {
16
```

Figura 3. Anotación `@CrossOrigin(origins = "**")` en el controlador.



```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE html>
<html><body>
  <pre id="out">Cargando...</pre>
  <script>
    fetch('http://localhost:8080/api/pacientes')
      .then(r => r.json())
      .then(data => document.getElementById('out').innerText = JSON.stringify(data, null, 2))
      .catch(e => document.getElementById('out').innerText = 'Error: ' + e);
  </script>
</body></html>
```

Figura 4. Página HTML servida desde un origen distinto (`file://` / otro puerto) que hace `fetch()` al API del hospital.



```
[
  {
    "id": 1,
    "nombre": "Juan",
    "apellido": "Perez",
    "fechaNacimiento": "1985-03-15",
    "email": "juan.perez@email.com",
    "telefono": "0991234567",
    "direccion": "Av. Amazonas N45-123",
    "activo": true
  },
  {
    "id": 2,
    "nombre": "Maria",
    "apellido": "Garcia",
    "fechaNacimiento": "1990-07-22",
    "email": "maria.garcia@email.com",
    "telefono": "0987654321",
    "direccion": "Calle Loja E4-56",
    "activo": true
  },
  {
    "id": 3,
    "nombre": "Carlos",
    "apellido": "Lopez",
    "fechaNacimiento": "1978-11-30",
    "email": "carlos.lopez@email.com",
    "telefono": "0976543210",
    "direccion": "Av. 6 de Diciembre N34-78",
    "activo": true
  },
  {
    "id": 4,
    "nombre": "Ana",
    "apellido": "Martinez",
    "fechaNacimiento": "2000-01-10",
    "email": "ana.martinez@email.com",
    "telefono": "0965432109",
    "direccion": "Calle Cuenca E2-14",
    "activo": true
  },
  {
    "id": 5,
    "nombre": "Pedro",
    "apellido": "Ramirez",
    "fechaNacimiento": "1965-06-05",
    "email": "pedro.ramirez@email.com",
    "telefono": "0954321098",
    "direccion": "Av. Colon O3-45",
    "activo": false
  }
]
```

Figura 5. Respuesta JSON del API obtenida exitosamente desde el origen externo, sin bloqueo de CORS por parte del navegador.

Verificación

- Se creó un archivo HTML simple servido desde un origen distinto al del backend (por ejemplo abierto directamente como `file://` o en otro puerto), con un script que ejecuta `fetch('http://localhost:8080/api/pacientes')`.
- Al abrir el archivo en el navegador, la respuesta del API se muestra completa en la página, sin el error de CORS que el navegador mostraría si el backend restringiera los orígenes permitidos.

Recomendación de mitigación

- Reemplazar `origins = "**"` por una lista explícita de dominios confiables (por ejemplo, el dominio real del frontend en producción).
- Centralizar la configuración de CORS en una clase `WebMvcConfigurer` en lugar de repetir la anotación en cada controlador, para mantenerla consistente y fácil de auditar.



- Combinar con autenticación (ver Hallazgo 3) para que, aunque un origen esté permitido, las peticiones sensibles sigan requiriendo credenciales válidas.

Hallazgo 3: Ausencia total de autenticación y autorización (Broken Access Control)

Ubicación	Todo el backend. Se confirmó en backend/pom.xml que no existe la dependencia spring-boot-starter-security, y no existe ninguna clase de configuración de seguridad en el proyecto.
Categoría OWASP	A01:2021 – Broken Access Control
Riesgo	Alto

Impacto potencial

Cualquier persona con acceso a la red donde corre el backend puede leer, crear, modificar o eliminar pacientes, doctores, citas e historias clínicas sin presentar ninguna credencial. En un sistema hospitalario real esto expone datos médicos sensibles (protegidos típicamente por normativas de privacidad) y permite manipulación no autorizada de la información clínica.

Evidencia

```
PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> curl.exe "http://localhost:8080/api/pacientes"
[{"id":1,"nombre":"Juan","apellido":"Perez","fechaNacimiento":"1985-03-15","email":"juan.perez@email.com","telefono":"0991234567","direccion":"Av. Amazonas N45-123","activo":true},
{"id":2,"nombre":"Maria","apellido":"Garcia","fechaNacimiento":"1990-07-22","email":"maria.garcia@email.com","telefono":"0987654321","direccion":"Calle Loja E4-56","activo":true},
{"id":3,"nombre":"Carlos","apellido":"Lopez","fechaNacimiento":"1978-11-30","email":"carlos.lopez@email.com","telefono":"0976543210","direccion":"Av. 6 de Diciembre N34-78","activo":true},
{"id":4,"nombre":"Ana","apellido":"Martinez","fechaNacimiento":"2000-01-10","email":"ana.martinez@email.com","telefono":"0965432109","direccion":"Calle Cuenca E2-14","activo":true},
{"id":5,"nombre":"Pedro","apellido":"Ramirez","fechaNacimiento":"1965-06-05","email":"pedro.ramirez@email.com","telefono":"0954321098","direccion":"Av. Colon O3-45","activo":false}]

PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> curl.exe -i -X DELETE "http://localhost:8080/api/pacientes/5"
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Length: 0
Date: Wed, 22 Jul 2026 20:12:46 GMT

PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> curl.exe "http://localhost:8080/api/pacientes"
[{"id":1,"nombre":"Juan","apellido":"Perez","fechaNacimiento":"1985-03-15","email":"juan.perez@email.com","telefono":"0991234567","direccion":"Av. Amazonas N45-123","activo":true},
{"id":2,"nombre":"Maria","apellido":"Garcia","fechaNacimiento":"1990-07-22","email":"maria.garcia@email.com","telefono":"0987654321","direccion":"Calle Loja E4-56","activo":true},
{"id":3,"nombre":"Carlos","apellido":"Lopez","fechaNacimiento":"1978-11-30","email":"carlos.lopez@email.com","telefono":"0976543210","direccion":"Av. 6 de Diciembre N34-78","activo":true},
{"id":4,"nombre":"Ana","apellido":"Martinez","fechaNacimiento":"2000-01-10","email":"ana.martinez@email.com","telefono":"0965432109","direccion":"Calle Cuenca E2-14","activo":true}]

PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> █
```

Figura 6. Petición a un endpoint sensible (por ejemplo eliminación o consulta de historia clínica) ejecutada exitosamente sin ningún header de autenticación.

```
PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> curl.exe -i -X POST "http://localhost:8080/api/pacientes" -H "Content-Type: application/json" -d '{"nombre":"Intruso","apellido":"Test"}'
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 22 Jul 2026 20:14:06 GMT

{"id":6,"nombre":"Intruso","apellido":"Test","fechaNacimiento":null,"email":null,"telefono":null,"direccion":null,"activo":true}

PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> curl.exe -i -X PUT "http://localhost:8080/api/doctores/1" -H "Content-Type: application/json" -d '{"nombre":"Hackeador","apellido":"Rodriguez","especialidad":"Cardiologia"}'
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 22 Jul 2026 20:14:26 GMT

{"id":1,"nombre":"Hackeador","apellido":"Rodriguez","especialidad":"Cardiologia","email":null,"telefono":null,"consultorio":null}

PS G:\EPN\Septimo\V&V\Practicas\hospital-management\backend> █
```

Figura 7. Prueba adicional sobre otra operación sensible, confirmando el mismo comportamiento: el sistema no exige ninguna credencial.



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
    <relativePath/>
  </parent>
  <groupId>com.hospital</groupId>
  <artifactId>hospital-management</artifactId>
  <version>1.0.0</version>
  <name>Hospital Management System</name>
  <description>Sistema de gestion hospitalaria - Proyecto de Validacion y Verificacion de Software</description>
```

Figura 8. pom.xml del backend, confirmando que no existe la dependencia spring-boot-starter-security ni ninguna librería de autenticación.

Verificación

- Se invocaron endpoints que modifican o exponen datos sensibles (creación, edición, eliminación de pacientes/doctores/citas/historias) sin enviar ningún header Authorization ni cookie de sesión, y todas las operaciones se completaron con código 200/201/204 en lugar de 401/403.
- Se revisó pom.xml y el código fuente completo del backend, confirmando la ausencia de spring-boot-starter-security, filtros de autenticación o cualquier SecurityConfig.

Recomendación de mitigación

- Incorporar Spring Security con un mecanismo de autenticación adecuado (JWT o sesión) y control de acceso basado en roles (por ejemplo, ADMIN vs DOCTOR).
- Proteger como mínimo los endpoints de escritura (POST/PUT/DELETE) y los que exponen historias clínicas, exigiendo un token válido.
- Aplicar el principio de menor privilegio: cada rol debe acceder únicamente a los recursos que necesita.

Hallazgo 4: Exposición de información sensible en errores (Stack Trace Disclosure)

Ubicación	backend/src/main/java/com/hospital/exception/GlobalExceptionHandler.java, método handleGeneral().
Categoría OWASP	A05:2021 – Security Misconfiguration (relacionado también con A09 – Security Logging and Monitoring Failures)
Riesgo	Medio

Impacto potencial

El stack trace completo revela nombres de clases internas, paquetes, versiones de librerías y estructura del código al cliente. Esta información facilita el reconocimiento (fingerprinting) por parte de un atacante, ayudándolo a identificar componentes vulnerables o a refinar otros ataques.

Evidencia



```

C:\Users\sepio\OneDrive\Practicas\hospital-management\backend> curl.exe -i "http://localhost:8080/api/pacientes/shc"
HTTP/1.1 500
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 22 Jul 2026 19:00:12 GMT
Content-Length: 4000

{"stackTrace":[{"className":"app","moduleName":"null","moduleVersion":"null","methodName":"convertIfNecessary","fileName":"AbstractNamedValueMethodArgumentResolver.java","lineNum
ber":302,"nativeMethod":false,"className":"org.springframework.web.method.annotation.AbstractNamedValueMethodArgumentResolver","classNameName":"app","moduleName":"null","moduleVe
rsion":"null","methodName":"resolveArgument","fileName":"AbstractNamedValueMethodArgumentResolver.java","lineNumber":137,"nativeMethod":false,"className":"org.springframework.web.meth
od.annotation.AbstractNamedValueMethodArgumentResolver","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"resolveArgument","fileName":"HandlerMethodAr
gumentResolverComposite.java","lineNumber":122,"nativeMethod":false,"className":"org.springframework.web.method.support.HandlerMethodArgumentResolverComposite","classNameName":"org
.springframework.web.method.support.support.InvokeableHandlerMethod","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"invokeForRequest","fileName":"InvokeableHan
dlerMethod.java","lineNumber":178,"nativeMethod":false,"className":"org.springframework.web.method.support.InvokeableHandlerMethod","classNameName":"app","moduleName":"null","moduleVer
sion":"null","methodName":"invokeAndHandle","fileName":"ServletInvokeableHandlerMethod.java","lineNumber":118,"nativeMethod":false,"className":"org.springframework.web.servlet.mvc.meth
od.annotation.ServletInvokeableHandlerMethod","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"invokeHandlerMethod","fileName":"RequestMappingAdapte
r.java","lineNumber":926,"nativeMethod":false,"className":"org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter","classNameName":"app","moduleNa
me":"null","moduleVersion":"null","methodName":"handleInternal","fileName":"RequestMappingHandlerAdapter.java","lineNumber":831,"nativeMethod":false,"className":"org.springfram
ework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"handle","fileName":"AbstractHan
dlerMethodAdapter.java","lineNumber":97,"nativeMethod":false,"className":"org.springframework.web.servlet.mvc.method.annotation.AbstractHandlerMethodAdapter","classNameName":"app","module
Name":"null","moduleVersion":"null","methodName":"doDispatch","fileName":"DispatcherServlet.java","lineNumber":1089,"nativeMethod":false,"className":"org.springframework.web.servet.Di
spatcherServlet","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"doService","fileName":"DispatcherServlet.java","lineNumber":979,"nativeMethod":false
,"className":"org.springframework.web.servlet.DispatcherServlet","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"processRequest","fileName":"Framework
Servlet.java","lineNumber":1014,"nativeMethod":false,"className":"org.springframework.web.servlet.FrameworkServlet","classNameName":"app","moduleName":"null","moduleVersion":"nul
l","methodName":"doGet","fileName":"FrameworkServlet.java","lineNumber":903,"nativeMethod":false,"className":"org.springframework.web.servlet.FrameworkServlet","classNameName":"a
pp","moduleName":"null","moduleVersion":"null","methodName":"service","fileName":"HttpServlet.java","lineNumber":564,"nativeMethod":false,"className":"jakarta.servlet.http.HttpServlet
","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"service","fileName":"FrameworkServlet.java","lineNumber":885,"nativeMethod":false,"className":"org
.springframework.web.servlet.FrameworkServlet","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"service","fileName":"HttpServlet.java","lineNumber":564
,"nativeMethod":false,"className":"jakarta.servlet.http.HttpServlet","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"internalDoFilter","fileName":"Ap
plicationFilterChain.java","lineNumber":195,"nativeMethod":false,"className":"org.apache.catalina.core.ApplicationFilterChain","classNameName":"app","moduleName":"null","moduleVer
sion":"null","methodName":"doFilter","fileName":"ApplicationFilterChain.java","lineNumber":140,"nativeMethod":false,"className":"org.apache.catalina.core.ApplicationFilterChain","cl
assLoaderName":"app","moduleName":"null","moduleVersion":"null","methodName":"doFilter","fileName":"WebFilter.java","lineNumber":51,"nativeMethod":false,"className":"org.apache.tomcat
websocket.server.WebFilter","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"internalDoFilter","fileName":"ApplicationFilterChain.java","lineNumber":1
64,"nativeMethod":false,"className":"org.apache.catalina.core.ApplicationFilterChain","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"doFilter","fileNa
me":"ApplicationFilterChain.java","lineNumber":140,"nativeMethod":false,"className":"org.apache.catalina.core.ApplicationFilterChain","classNameName":"app","moduleName":"null","moduleVe
rsion":"null","methodName":"doFilter","fileName":"RequestDispatcherFilter.java","lineNumber":190,"nativeMethod":false,"className":"org.springframework.web.filter.RequestDis
patcherFilter","classNameName":"app","moduleName":"null","moduleVersion":"null","methodName":"doFilter","fileName":"OncePerRequestFilter.java","lineNumber":116,"nativeMethod":fal
se,"className":"org.springframework.web.filter.OncePerRequestFilter"}]}

```

- Se provocó intencionalmente un error 500 (por ejemplo, enviando un identificador no numérico donde se espera un Long, o un JSON malformado en el cuerpo de la petición).
- La respuesta JSON del servidor incluyó el campo stackTrace con las clases, métodos y números de línea internos del backend.

9



- Eliminar el campo stackTrace de la respuesta al cliente; devolver únicamente un mensaje genérico ("Ocurrió un error interno").
- Registrar el stack trace completo del lado del servidor usando un logger (SLF4J/Logback) en lugar de exponerlo al cliente.
- Diferenciar el nivel de detalle del error según el perfil (dev vs prod), nunca exponiendo detalles internos en producción.

Hallazgo 5: Credenciales de base de datos en texto plano

Ubicación	backend/src/main/resources/application.properties — spring.datasource.password=hospital123
Categoría OWASP	A02:2021 – Cryptographic Failures (gestión insegura de credenciales/secrets)
Riesgo	Medio

Impacto potencial

La contraseña de la base de datos queda expuesta en texto plano dentro del repositorio de código fuente. Cualquiera con acceso al repositorio (incluyendo su historial de Git) puede obtener credenciales válidas para conectarse directamente a la base de datos, saltándose por completo la aplicación.

Evidencia

```
# PostgreSQL Connection
spring.datasource.url=jdbc:postgresql://localhost:5432/hospital
spring.datasource.username=admin
spring.datasource.password=hospital123
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
1  # =====
2  # Hospital Management System - Configuración
3  # =====
4
5  # PostgreSQL Connection
6  spring.datasource.url=jdbc:postgresql://localhost:5432/hospital
7  spring.datasource.username=admin
8  spring.datasource.password=hospital123
9  spring.datasource.driver-class-name=org.postgresql.Driver
10
```

Figura 11. application.properties con la contraseña de la base de datos en texto plano.

```
# PostgreSQL Connection
spring.datasource.url=jdbc:postgresql://localhost:5432/hospital
spring.datasource.username=admin
spring.datasource.password=hospital123
spring.datasource.driver-class-name=org.postgresql.Driver
```



Figura 12. Evidencia adicional de las credenciales expuestas en el archivo de configuración versionado.

Verificación

- Se localizó el archivo `application.properties` dentro del repositorio y se confirmó que contiene usuario y contraseña de PostgreSQL en texto plano.
- Se verificó que, con esas credenciales, es posible conectarse directamente a la base de datos (por ejemplo con `docker exec -it hospital-db psql -U admin -d hospital`) sin pasar por la aplicación.

Recomendación de mitigación

- Externalizar las credenciales usando variables de entorno (`spring.datasource.password=${DB_PASSWORD}`) inyectadas en tiempo de despliegue.
- Excluir los archivos de configuración con secretos del control de versiones (`.gitignore`) o usar un gestor de secretos (Vault, AWS Secrets Manager, variables de entorno del contenedor).
- Rotar la contraseña actual, ya que quedó expuesta en el historial del repositorio.

Hallazgo 6: Cross-Site Scripting (XSS) vía innerHTML sin sanitizar

Ubicación	frontend/js/historias.js (guardar diagnóstico y verHistoria, ~líneas 130 y 147), frontend/js/pacientes.js (renderTabla, ~línea 49), frontend/js/doctores.js (renderTabla), frontend/js/utiles.js (showAlert, ~línea 70, y escapeHTML con escape incompleto, ~línea 51).
Categoría OWASP	A03:2021 – Injection (Cross-Site Scripting)
Riesgo	Alto

Impacto potencial

Al insertar datos del usuario o del backend directamente con `innerHTML` sin sanitizar, un atacante puede almacenar JavaScript malicioso en campos como diagnóstico, nombre o mensaje de alerta. Ese script se ejecuta en el navegador de cualquier persona que visualice el dato (robo de sesión, redirección a sitios maliciosos, o manipulación de la interfaz).

Evidencia

```
// historias.js
// BUG INTENCIONAL: No sanitiza el diagnostico, permitiendo XSS almacenado
// Un usuario malicioso puede ingresar: <script>alert('XSS')</script>

// BUG INTENCIONAL: innerHTML con datos del backend sin sanitizar (XSS)
const detalle = `<div>...${h.diagnostico}...</div>`;
contenedor.innerHTML = detalle;
```



```
JS citas.js x
11 const CitasModule :({}) = {
47   renderTabla(citas) :void {
54     tbody.innerHTML = citas.map(c => {
55       // BUG INTENCIONAL: doctor puede ser null y causa error en render
56       const doctorNombre :string = c.doctor
57       ? `${c.doctor.nombre} ${c.doctor.apellido}`
58       : 'No asignado';
59
60       return `
61         <tr>
62           <td>Paciente #${c.pacienteId}</td>
63           <td>${doctorNombre}</td>
64           <td>${formatDateTime(c.fechaHora)}</td>
65           <td>${c.motivo || '-'}</td>
66           <td><span class="badge badge-${(c.estado || '').toLowerCase()}>${c.estado}</span></td>
67           <td class="actions">
68             <button class="btn-edit" onclick="CitasModule.editarCita(${c.id})">Editar</button>
69             <button class="btn-delete" onclick="CitasModule.eliminarCita(${c.id})">Eliminar</button>
70           </td>
71         </tr>
72       `;
73     }).join('');
74   },
75 }
```

Figura 13. Render de historias clínicas usando innerHTML con el diagnóstico sin escapar.

```
JS pacientes.js x
11 const PacientesModule :({}) = {
42   renderTabla(pacientes) :void {
48
49     // BUG INTENCIONAL: usa innerHTML con datos que podrían tener XSS
50     // (aunque el backend no sanitiza, el frontend tampoco)
51     tbody.innerHTML = pacientes.map(p => `
52       <tr>
53         <td>${p.nombre} ${p.apellido}</td>
54         <td>${p.email || '-'}</td>
55         <td>${p.telefono || '-'}</td>
56         <td>${formatDate(p.fechaNacimiento)}</td>
57         <td><span class="badge ${p.activo ? 'badge-activo' : 'badge-inactivo'}>${p.activo ? 'Activo' : 'Inactivo'}</span></td>
58         <td class="actions">
59           <button class="btn-view" onclick="PacientesModule.verPaciente(${p.id})">Ver</button>
60           <button class="btn-edit" onclick="PacientesModule.editarPaciente(${p.id})">Editar</button>
61           <button class="btn-delete" onclick="PacientesModule.eliminarPaciente(${p.id})">Eliminar</button>
62         </td>
63       </tr>
64     `).join('');
65   },
66 }
```

Figura 14. Render de la tabla de pacientes, también con innerHTML sin sanitizar.

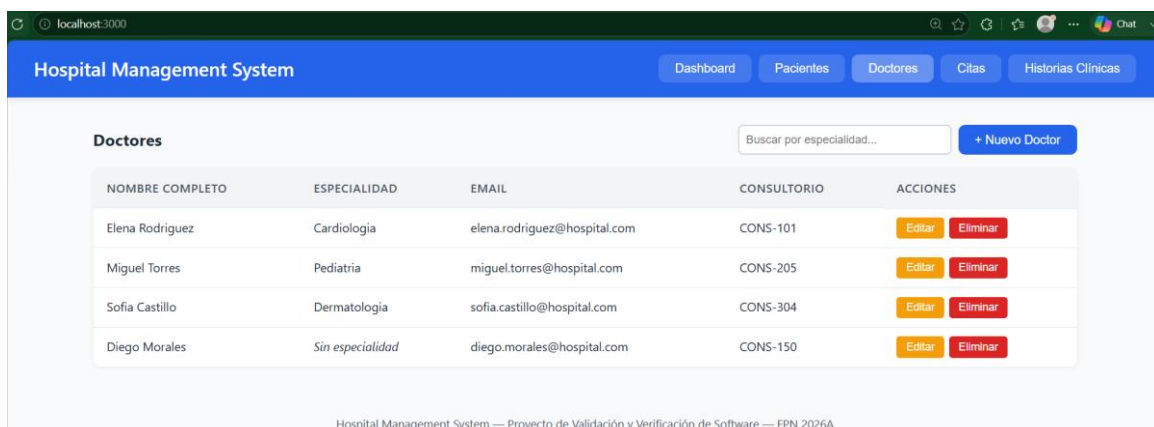


Figura 15. Formulario donde se ingresó un payload de prueba tipo `<script></img onerror=...>` como diagnóstico.



Formulario de Doctor

Nombre *

Apellido *

Especialidad

Email

Teléfono

Consultorio

Figura 16. El payload se ejecuta al visualizar el registro guardado.

```
::1 - [22/Jul/2026 14:08:51] code 404, message File not found
::1 - [22/Jul/2026 14:08:51] "GET /x HTTP/1.1" 404 -
::1 - [22/Jul/2026 14:09:40] code 404, message File not found
::1 - [22/Jul/2026 14:09:40] "GET /x HTTP/1.1" 404 -
::1 - [22/Jul/2026 14:09:47] code 404, message File not found
::1 - [22/Jul/2026 14:09:47] "GET /x HTTP/1.1" 404 -
::1 - [22/Jul/2026 14:09:59] code 404, message File not found
::1 - [22/Jul/2026 14:09:59] "GET /x HTTP/1.1" 404 -
```

Figura 17. Evidencia adicional de la ejecución del script inyectado.

The screenshot shows the 'Hospital Management System' interface. On the left, a table lists doctors with columns: NOMBRE COMPLETO, ESPECIALIDAD, EMAIL, and CONSULTORIO. The table includes entries for Elena Rodriguez, Miguel Torres, Sofia Castillo, and Diego Morales. A 'Test XSS' entry is also present. On the right, a web browser console shows a 404 error message: 'code 404, message File not found' for the request 'GET /x HTTP/1.1'.

Figura 18. Confirmación en consola/logs de la petición disparada por el payload ejecutado (ej. llamada a /x generada por el script inyectado).

Verificación

- Se creó una historia clínica (o paciente) usando como diagnóstico/nombre un payload de prueba, por ejemplo: `<img src=x onerror=fetch('/x')>` o `<script>alert('XSS')</script>`.



- Al listar o abrir el detalle del registro guardado, el payload se ejecutó en el navegador, confirmado por la alerta/petición disparada y visible en los logs del servidor de archivos estáticos.

Recomendación de mitigación

- Reemplazar innerHTML por textContent cuando el contenido es texto plano, o usar una librería de sanitización como DOMPurify antes de insertar HTML dinámico.
- Completar la función escapeHTML en utils.js para escapar también comillas simples (') y backticks (`), y aplicarla de forma consistente en todos los módulos (historias.js, pacientes.js, doctores.js, citas.js).
- Sanitizar también en el backend antes de persistir el diagnóstico/observaciones, para no depender únicamente de la validación del frontend.

Hallazgo 7: Falta de restricción NOT NULL en especialidad de doctores

Ubicación	backend/src/main/resources/schema.sql (tabla doctores, columna especialidad) y backend/src/main/java/com/hospital/dto/DoctorDTO.java (sin @NotBlank).
Categoría OWASP	A04:2021 – Insecure Design (validación de datos insuficiente en el diseño del esquema)
Riesgo	Bajo

Impacto potencial

Aunque la especialidad es un campo requerido según la lógica de negocio (HU-02), ni la base de datos ni el DTO lo obligan. Esto permite datos inconsistentes que pueden romper búsquedas por especialidad o generar comportamiento inesperado en el frontend (por ejemplo, mostrar "undefined" o vacío en reportes).

Evidencia

```
-- schema.sql
especialidad VARCHAR(100), -- BUG: deberia ser NOT NULL
```

```
-- Tabla: citas
CREATE TABLE citas (
    id BIGSERIAL PRIMARY KEY,
    paciente_id BIGINT NOT NULL,
    doctor_id BIGINT NOT NULL,
    fecha_hora TIMESTAMP NOT NULL,
    motivo VARCHAR(255),
    estado VARCHAR(20) DEFAULT 'PROGRAMADA',
    -- Correccion estructural: Llave foranea hacia pacientes agregada
    CONSTRAINT fk_citas_paciente FOREIGN KEY (paciente_id) REFERENCES pacientes(id),
    CONSTRAINT fk_citas_doctor FOREIGN KEY (doctor_id) REFERENCES doctores(id)
);
```

Figura 19. Registro real en la base de datos (doctor Diego Morales) con el campo especialidad en NULL, confirmando que la falta de restricción permite datos incompletos.

Verificación — cómo comprobarlo

- Se consultó el endpoint GET /api/doctores y se observó que el doctor "Diego Morales" (dato semilla del propio sistema) tiene especialidad: null, lo que confirma que el campo puede quedar vacío sin que la base de datos ni el DTO lo impidan.

Recomendación de mitigación



- Agregar la restricción NOT NULL (o un CHECK que valide que no esté vacío) a la columna especialidad en schema.sql.
- Agregar @NotBlank en DoctorDTO para que la validación también se aplique en la capa de entrada del API antes de llegar a la base de datos.

Hallazgo 8: División por cero en el cálculo de edad promedio (hallazgo adicional)

Ubicación	backend/src/main/java/com/hospital/service/PacienteService.java, método calcularEdadPromedio().
Categoría OWASP	A04:2021 – Insecure Design (manejo inadecuado de un caso límite, con potencial de interrupción del servicio)
Riesgo	Bajo

Impacto potencial

Si no existen pacientes registrados (lista vacía), el cálculo `suma / pacientes.size()` produce una división por cero, lo que puede provocar una respuesta de error inesperada en el endpoint de estadísticas del dashboard, afectando la disponibilidad de esa funcionalidad.

Evidencia

```
public double calcularEdadPromedio() {  
    List<Paciente> pacientes = pacienteRepository.findAll();  
    long suma = 0;  
    for (Paciente p : pacientes) {  
        if (p.getFechaNacimiento() != null) {  
            suma += Period.between(p.getFechaNacimiento(), LocalDate.now()).getYears();  
        }  
    }  
    return (double) suma / pacientes.size(); // BUG: division by zero si lista vacia  
}
```

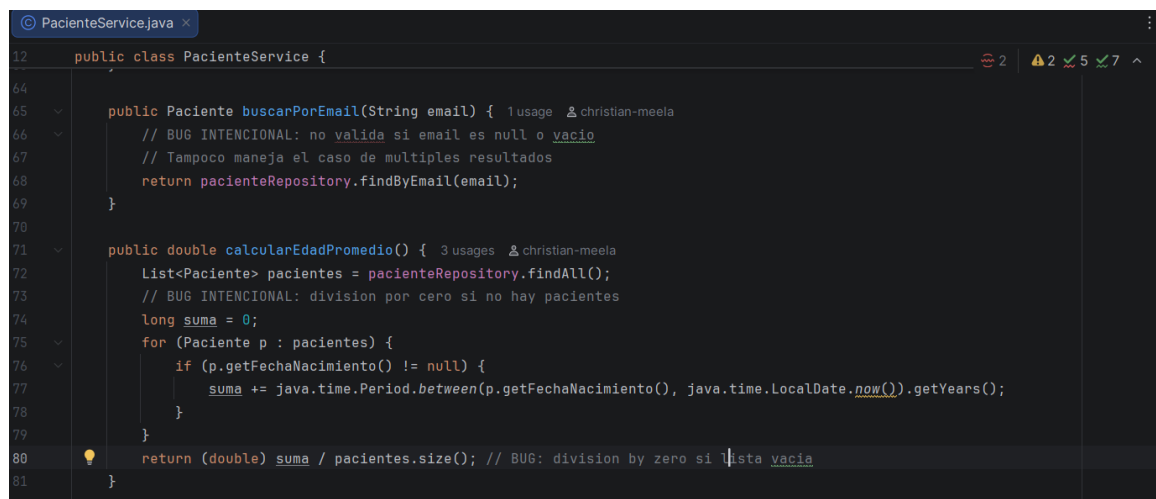


Figura 20. Código del cálculo de edad promedio sin validar lista vacía.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS G:\EPN\Septimo\VBV\Practicas\hospital-management\backend> docker exec -it hospital-db psql -U admin -d hospital -c "DELETE FROM historias_clinicas; DELETE FROM citas; DELETE FROM pacientes;"
DELETE 3
DELETE 5
DELETE 5

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image -> docker debug hospital-db
Learn more at https://docs.docker.com/go/debug-cli/

PS G:\EPN\Septimo\VBV\Practicas\hospital-management\backend> curl.exe -i "http://localhost:8080/api/pacientes/estadisticas/edad-promedio"
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 22 Jul 2026 19:23:59 GMT

"NaN"
```

Figura 21. Resultado observado al invocar el endpoint de estadísticas de edad promedio sin pacientes registrados.

Verificación

- Se vació o filtró la tabla de pacientes (o se probó en un entorno sin datos) y se invocó GET /api/pacientes/estadisticas/edad-promedio, observando el resultado anómalo (NaN/Infinity o error) en lugar de un valor numérico válido o un mensaje controlado.

Recomendación de mitigación

- Validar si la lista de pacientes está vacía antes de dividir, retornando 0 o un valor/mensaje explícito en ese caso.
- Agregar una prueba unitaria de caso límite (lista vacía) para este método, como exige además el entregable 7.1 de pruebas unitarias.

5. Conclusiones

- El sistema Hospital Management System presenta vulnerabilidades críticas de las categorías A01 (Broken Access Control) y A03 (Injection), que en un entorno de producción comprometerían por completo la confidencialidad e integridad de los datos clínicos.
- La ausencia total de autenticación es, en conjunto con la inyección SQL y el XSS, el hallazgo de mayor riesgo: cualquier persona con acceso a la red puede leer o modificar historias clínicas sin ninguna barrera.
- Los hallazgos de configuración (CORS abierto, stack trace expuesto, credenciales en texto plano) agravan el impacto de las vulnerabilidades críticas al facilitar su descubrimiento y explotación desde fuera de la red local.
- Se recomienda priorizar, en este orden, la implementación de autenticación/autorización (Spring Security), la corrección de las consultas SQL concatenadas, y la sanitización sistemática de las salidas HTML en el frontend.
- El análisis confirma que las prácticas de validación y verificación aplicadas (revisión de código + pruebas dinámicas) permiten detectar y comprobar de forma reproducible vulnerabilidades reales del estándar OWASP Top 10 (2021).